Vulnerability Assessment of the ODTN Cyberattack Crisis Simulator

Cyberattack Cripples Transit – Commuters Stranded
Course: CYB
October 23, 2026.
OLAKUNLE

Post-Deployment Vulnerability Assessment

1. Executive Summary

The purpose of this vulnerability assessment is to review the security posture of my ODTN interactive cyberattack crisis simulator, titled "Cyberattack Cripples Transit – Commuters Stranded." The project simulates a cyberattack against a public transportation system and allows users to make incident response decisions. The website was developed to demonstrate cybersecurity concepts, including incident response, network security, threat containment, and business continuity.

The assessment focuses on potential security weaknesses that may occur in a web application developed and deployed with the assistance of artificial intelligence (AI). These include exposed API keys, improper input validation, authentication weaknesses, front-end vulnerabilities, security misconfigurations, unsafe AI integrations, exposed project files, and unnecessary attack surface.

Since this assessment is based on the project design rather than a verified technical scan, the findings are preliminary and require further testing. The main security priorities are to ensure that no sensitive information is exposed, user inputs are handled safely, administrative functions are protected, and the website does not expose unnecessary files or services.

Overall, the project provides a useful cybersecurity learning experience, but it should not be considered secure for production use until its security controls have been verified. Applying secure coding practices, reviewing AI-generated code, and conducting regular vulnerability assessments would help reduce the risk of attacks.

2. Project Overview

The ODTN crisis simulator is an interactive web experience based on a fictional cyberattack that disrupts public transportation services and leaves commuters stranded. Users act as members of an incident response team and make decisions about responding to the attack, containing threats, protecting systems, communicating with the public, and restoring services.

The project demonstrates how cybersecurity incidents can affect essential services. However, because the website is publicly accessible after deployment, it must also be reviewed for security weaknesses that could affect its users or hosting environment.

3. Assessment Methodology

The assessment uses a defensive security review approach. The following areas should be examined:

1. Source code review: Check HTML, CSS, JavaScript, configuration files, and dependencies for exposed secrets or unsafe coding practices.
2. Input validation: Test whether text fields and other user inputs are handled safely.
3. Authentication and access control: Determine whether restricted pages or administrative functions are properly protected.
4. Deployment configuration: Review HTTPS, security headers, directory listings, error messages, and exposed files.
5. AI integration: Check whether AI services use protected API keys and whether user input is handled safely.
6. Attack surface review: Identify unnecessary endpoints, services, scripts, and third-party dependencies.

Only systems and accounts that I own or have permission to test should be included in the assessment.

4. Key Findings and Severity Ratings

The following findings are potential vulnerabilities to investigate. Their actual presence and severity must be confirmed through testing.

Finding 1: Exposed API Keys or Secrets

Severity: High if confirmed; otherwise, unverified

API keys, passwords, access tokens, or other credentials may accidentally be included in JavaScript files, source code, or public repositories during development.

If an attacker obtains a valid secret, they may be able to access connected services, misuse an API, or generate unauthorized charges.

Recommended remediation:

Search the source code and repository history for API keys, tokens, passwords, and other secrets.
Store server-side credentials in environment variables or a dedicated secrets manager.
Never place private API keys in browser-side JavaScript.
Revoke and rotate any exposed credentials.
Use restricted API permissions and usage limits.

Finding 2: Input Validation Weaknesses

Severity: Medium if confirmed

If the simulator accepts user input through text fields, forms, or interactive components, the application must handle that input safely. Poor validation could allow malicious input to affect the page or application.

For example, unsafe rendering of user-supplied HTML may create a cross-site scripting (XSS) vulnerability.

Recommended remediation:

Validate user input on the server whenever a server processes it.
Use context-appropriate output encoding.
Prefer safe DOM methods such as `textContent` instead of inserting untrusted content using `innerHTML`.
Avoid evaluating user input as JavaScript.
Test inputs containing HTML characters and script-like strings in an authorized test environment.

Finding 3: Authentication and Access Control Weaknesses

Severity: High if sensitive functions are exposed; otherwise, unverified

If the simulator contains only public educational content, authentication may not be necessary. However, if it includes an administrator page, saved user information, private reports, or restricted functions, those features require appropriate access controls.

Hiding a button or page in the front end does not provide sufficient protection if the underlying resource remains accessible.

Recommended remediation:

Require authentication for restricted functions.
Enforce authorization on the server, not only in the browser.
Apply the principle of least privilege.
Use secure session management and appropriate cookie settings if login functionality exists.
Avoid collecting personal information unless it is necessary.

Finding 4: Front-End Security Vulnerabilities

Severity: Medium if confirmed

The simulator may use JavaScript to display incident scenarios, process user choices, and update the interface. Front-end code is accessible to website visitors, so it must not contain confidential information or rely on the browser to enforce security controls.

Unsafe JavaScript functions or vulnerable third-party scripts could introduce additional risks.

Recommended remediation:

Review JavaScript for unsafe DOM manipulation and dynamic code execution.
Remove unnecessary scripts and third-party libraries.
Keep dependencies updated.
Use a Content Security Policy (CSP) to restrict permitted scripts and other resources.
Never store passwords, private keys, or other secrets in browser-accessible code.

Finding 5: Security Misconfiguration

Severity: Medium to High, depending on the exposure

Incorrect hosting settings may expose configuration files, directory listings, development tools, detailed error messages, or unnecessary services. Missing HTTPS or security headers could also weaken the website's protection.

Recommended remediation:

Enforce HTTPS.
Disable directory listings and unnecessary services.
Remove development and debugging features from production.
Configure appropriate security headers, including CSP and `X-Content-Type-Options`.
Configure `Strict-Transport-Security` when HTTPS is correctly established for the domain.
Review hosting permissions and production settings.

Finding 6: Unsafe AI Integrations

Severity: High if an AI service is connected insecurely; otherwise, unverified

AI-assisted development can introduce security risks if generated code connects to an AI service without protecting credentials or validating inputs. If the simulator does not use an AI API, this particular risk may not apply.

If an AI integration exists, malicious prompts or untrusted data could potentially influence its output. An exposed API key could also be misused.

Recommended remediation:

Keep private AI API keys on the server.
Validate and limit user input before sending it to an AI service.
Treat AI-generated responses as untrusted data.
Do not execute AI-generated code automatically.
Apply rate limits, usage limits, and appropriate access controls.
Avoid sending confidential information to external AI services without authorization.

Finding 7: Public Repository or Sensitive File Exposure

Severity: Medium to High, depending on the information exposed

A public code repository is not automatically a vulnerability. However, problems may arise if it contains private credentials, personal information, deployment configuration, internal documentation, or sensitive backup files.

Files such as `. env`, private configuration files, and database backups should not be publicly accessible.

Recommended remediation:

Review the repository's visibility and contents.
Add sensitive files to gitignore` before committing.
Check repository history for previously committed secrets.
Remove public access to sensitive files and rotate exposed credentials.
Keep only necessary public documentation and application files in the deployed site.

Finding 8: Excessive Attack Surface

Severity: Medium if unnecessary exposure is confirmed

Every additional endpoint, dependency, script, service, or external integration can introduce another possible entry point for an attacker. Unnecessary features may increase the maintenance burden and create opportunities for exploitation.

****Recommended remediation:**

Remove unused scripts, libraries, endpoints, and features.
Update vulnerable dependencies.
Limit external connections to trusted services.
Review network exposure and hosting permissions.
Monitor relevant security advisories and periodically reassess the application.

5. Severity Summary

Finding	Preliminary severity	Priority
Exposed API keys or secrets	High if confirmed	Immediate
Input validation weaknesses	Medium if confirmed	High
Authentication and access Control	High if sensitive functions exposed	Immediate if comfirmed
Front-end vulnerabilities	Medium if confirmed	High
Security misconfiguration	Medium–High	High
Unsafe AI integrations	High if insecure integration Exists	Immediate if comfirmed

Public repository or sensitive	Medium–High	High files
Excessive attack surface	Medium if unnecessary exposure exists	Moderate.

These ratings indicate potential impact, not verified vulnerabilities. The final ratings should be adjusted according to the actual code, deployment configuration, likelihood of exploitation, and sensitivity of the affected resources.

6. Recommended Remediation Plan

I would address the potential risks in the following order.

Priority 1: Protect Sensitive Information

Search the source code and repository for exposed secrets.
Revoke and replace any compromised credentials.
Remove confidential files from public access.

Priority 2: Secure the Application

Review input handling and unsafe JavaScript.
Verify authentication and authorization wherever restricted functions exist.
Confirm that HTTPS is enforced and production settings are secure.

Priority 3: Review AI and Third-Party Components

Inspect AI-generated code rather than trusting it automatically.
Verify that AI API credentials are not exposed.
Update dependencies and remove unnecessary integrations.

Priority 4: Test and Monitor

Run an authorized vulnerability scan against the deployed website.
Review browser developer tools and network requests for unexpected exposure.
Check server and hosting logs when available.
Repeat security reviews after major changes.

7. Reflection: How AI-Assisted Development Impacted Security

AI-assisted development can make web development faster and easier, especially for students who are still learning programming and cybersecurity. AI tools can help generate HTML, CSS, JavaScript, interactive features, and explanations of technical concepts. For my ODTN crisis simulator, AI assistance can help turn a cybersecurity scenario into an interactive learning experience.

However, AI-generated code is not automatically secure. It may contain outdated practices, unsafe input handling, unnecessary dependencies, or incorrect assumptions about authentication and access control. AI can also generate code that works correctly from a user's perspective while overlooking security requirements.

One important lesson is that a website can look professional and function properly while still having hidden vulnerabilities. For example, an application might display a working interface but expose a private API key in its JavaScript code. Similarly, hiding an administrative feature in the interface does not guarantee that unauthorized users cannot access its underlying functions.

AI should therefore be treated as a development assistant rather than a replacement for security knowledge and testing. Developers must review generated code, understand how data moves through the application, verify third-party components, and test security controls before deployment.

This assessment has helped me understand that cybersecurity must be considered throughout the software development lifecycle. A vulnerability assessment after deployment is valuable, but identifying security risks during design and development can reduce the chance of exposing weaknesses in the first place.

8. Conclusion

The ODTN cyberattack crisis simulator demonstrates how a cyber incident can disrupt public transportation and why effective incident response is important. However, the security of the website itself must also be considered.

The assessment identified eight areas requiring review, including exposed secrets, input validation, authentication, front-end security, misconfiguration, AI integration, public repository exposure, and attack surface management. These are potential risks that must be verified before being reported as confirmed vulnerabilities.

My main recommendation is to review the source code, secure any sensitive information, verify the deployment configuration, test the application's security controls, and maintain regular updates. AI-assisted development can improve productivity, but human review and security testing remain essential.

Ultimately, this assessment has reinforced the importance of building, deploying, and maintaining web applications securely rather than focusing only on their functionality and appearance.

Thanks